L5: Database Design using ERD (C6)

Md. Tariquzzaman
Junior Lecturer, Dept. of Computer Science and Engineering
Islamic University of Technology (IUT)

CSE 4307: Database Management Systems

Chapter Outline

- Overview of the Design Process
- The Entity-Relationship (ER) Model
- Mapping Cardinality and Constraints
- Weak and Strong Entity Sets
- Generalization and Specialization
- ER Diagram Implementation (Conversion to Tables)

Overview of the Design Process

Three Main Phases of Design

Database design is not a single step. It involves a sequence of phases:

1. **Initial Phase**: Characterize the data needs of users.
   - Requires extensive interaction with domain experts.
   - **Outcome**: A specification of user requirements.
2. **Conceptual-Design Phase**: Translating requirements into a schema.
   - Typically uses the **ER Model**
   - Abstract design; independent of specific database implementation.
3. **Implementation Phase**: Moving to the actual system.

The Implementation Phase

The implementation phase is further divided into two levels:

- **Logical Design**:
  - Maps the high-level conceptual schema onto the implementation data model (e.g., Relational Model/Tables).
- **Physical Design**:
  - Handles actual storage features.
  - Includes file organization, default space, and indexing.

Design Goals: Avoiding Pitfalls

A good database design must avoid two major problems:

- **Redundancy**:
  - Repeating information leads to wasted space.
  - **Risk**: Copies become inconsistent if updated separately.
- **Incompleteness**:
  - A bad design may make certain aspects of the enterprise impossible to model.
  - Introduces bad business logic.

Design Approaches

- Entity Relationship Model (will be covered in this chapter).
  - Collection of entities and relationships
  - Entity is an object (with a number of attributes), while relationship defines how entities are related.
  - Represented diagrammatically by an entity-relationship diagram
- Normalization Theory: Formal method to test if the design is good or bad. (will be covered in Chapter 7)

The Entity-Relationship (ER) Model

1. What is an Entity?

- **Definition**: An entity is an object that exists in the real world and is **distinguishable** from all other objects.
- **Types of Entities**:
  - **Concrete**: Tangible objects (e.g., a specific person like “Einstein”, a specific machine, an employee).
  - **Abstract**: Intangible concepts or events (e.g., a specific course like “CSE 4307”, a project, a flight booking).
- **Key Concept**: Distinguishability. Even if two entities appear identical (e.g., two students named ”John Smith”), they are distinct distinct individual entities.

2. Entity Sets

- **Definition**: An entity set is a collection of entities of the **same type** that share the same properties (attributes).
- **Analogy**:
  - If an **Entity** is like a “Row” in a table...
  - ...an **Entity Set** is the “Table” itself.
- **Examples**:
  - The set of all `Persons` in a university.
  - The set of all `Companies` in a stock market.
  - The set of all `Trees` in a forest.

2. Entity Sets

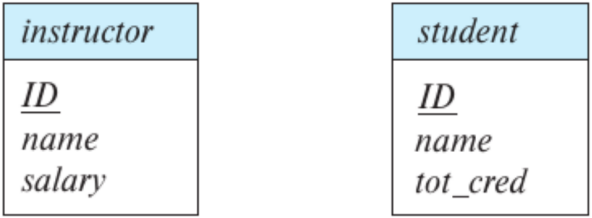


Figure: E-R diagram for entity sets instructor and student

3. Attributes: Defining the Entity

Entities are represented by a set of **Attributes** (descriptive properties).

Example 1: Instructor

An instructor is described by:

- `ID` (Unique Identifier)
- `name`
- `salary`

```
instructor = (ID, name, salary)
```

Example 2: Course

A course is described by:

- `course_id` (Unique Identifier)
- `title`
- `credits`

```
course = (course_id, title,
credits)
```

4. The Primary Key

- **Problem**: How do we distinguish between two entities if they share common values (e.g., two instructors with the same name and salary)?
- **Solution**: We define a **Primary Key**.
- **Definition**: A subset of attributes that **uniquely identifies** each member of the entity set.
- **Requirement**: No two entities in the set can have the same value for the primary key.

5. Visual Notation in ER Diagrams

In an ER Diagram, we use specific shapes to represent these concepts

- **Entity Sets**: Represented by **Rectangles**.
- **Attributes**: Listed **inside** the rectangle.
- **Primary Key**: Indicated by **underlining** the attribute name.

See the example on the next slide.

Example: Visual Representation

Here is how we represent the `instructor` and `student` entity sets in an ERD:

Entity Set: Instructor

1. **Rectangle** labeled "instructor"
2. **Attributes**:
   - <u>ID</u> (Underlined = PK)
   - name
   - salary

Entity Set: Student

1. **Rectangle** labeled "student"
2. **Attributes**:
   - <u>ID</u> (Underlined = PK)
   - name
   - tot_cred

Attributes

Attributes are descriptive properties possessed by all members of an entity set.

- **Simple Attributes**: Have no sub-parts (e.g., Salary).
- **Composite Attributes**: Can be divided into sub-parts
- **Single-valued**: Holds one value for an entity (e.g., Student ID)
- **Multi-valued**: Can have zero, one, or more values
- **Derived**: Value is computed, not stored (e.g., Age)

Example: Composite Attributes

Composite attributes are useful when different components are needed for different reports (e.g., sorting by Last Name or filtering by City).

Attribute: Name

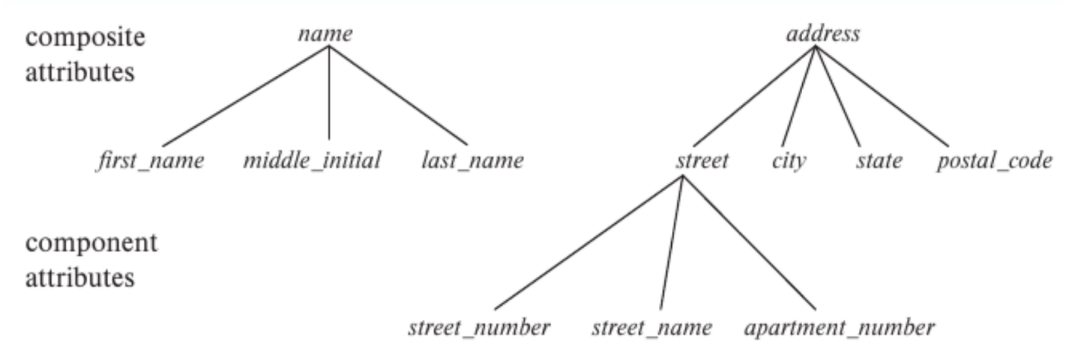
- First Name
- Middle Initial
- Last Name

Attribute: Address

- Street (Number, Name, Apt)
- City
- State
- Postal Code

Composite Attribute

Simple attribute has no subparts. While Composite attribute has subparts (i.e. Name= first name, last name). Composite attribute may appear as a hierarchy. (each level is called component attribute)



Example: Multi-valued and Derived Attributes

Multi-valued Attribute: Phone Number

- A single person may have:
  - Mobile: 017...
  - Home: 02...
  - Work: 019...
- *ER Notation*: Double Ellipse.

Derived Attribute: Age

- `Age` is calculated from `Date_of_Birth` and `Current_Date`.
- *ER Notation*: Dashed Ellipse.

Implementing Composite Attributes

In PostgreSQL, composite attributes are implemented by defining a `COMPOSITE TYPE`.

```sql
-- Step 1: Create the composite type
CREATE TYPE nametype AS (
fname VARCHAR(10),
middlename VARCHAR(10),
lastname VARCHAR(10)
);

-- Step 2: Use the type in a table
CREATE TABLE emp (
id INT PRIMARY KEY,
name nametype,
address VARCHAR(20)
);
```

Inserting Data into Composite Attributes

Once the table is created with a composite type, you can insert data using the `ROW` constructor or a tuple literal.

```sql
-- Step 3: Insert data using ROW() constructor
INSERT INTO emp VALUES
(1, ROW('John', 'Q', 'Public'), '123 Main St');

-- Alternative: Using simple tuple syntax
INSERT INTO emp VALUES
(2, ('Jane', NULL, 'Doe'), '456 Oak Ave');

-- Step 4: Accessing specific sub-fields
-- Note: Parentheses around the column name are required
SELECT id, (name).fname, (name).lastname
FROM emp;
```

Implementing Multi-valued Attributes

Multi-valued attributes are natively supported in PostgreSQL using **Arrays**.

```sql
-- Step 1: Create table with an ARRAY column
CREATE TABLE students_multiple (
id INT,
name VARCHAR(20),
phone VARCHAR(20)[]  -- The [] denotes an array
);

-- Inserting data
INSERT INTO students_multiple VALUES
(1, 'Alice', ARRAY['01711...', '01911...']);

-- Selecting data requires the unnest function
SELECT id, name, unnest(phone) FROM students_multiple;
```

E-R diagram of the attributes

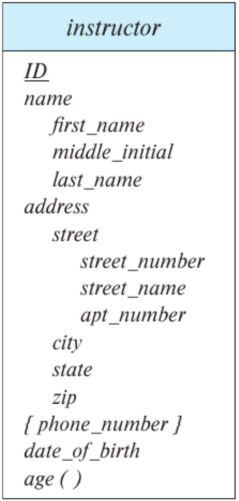


Figure: E-R diagram with composite, multivalued, and derived attributes

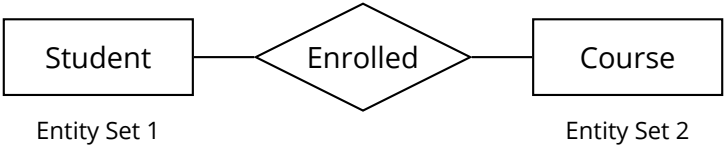
Relationships

Relationship Sets

- **Relationship**: An association among several entities.
- **Relationship Set**: A mathematical relation among $n \geq 2$ entity sets.
- **Degree**: The number of entity sets participating in a relationship.
  - **Binary**: Involves two entity sets (most common).
  - **Ternary**: Involves three entity sets.

Degree of Relationship: Binary

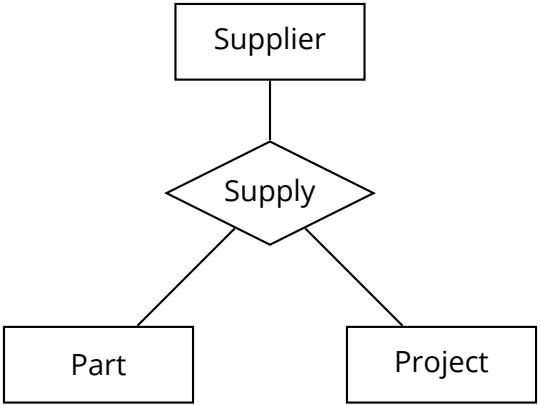
- **Binary Relationship**: Involves exactly **two** entity sets.
- This is the most common type of relationship in database systems.



Example: A Student takes a Course.

Degree of Relationship: Ternary

- **Ternary Relationship**: Involves **three** entity sets simultaneously.
- Used when the relationship cannot be split into binary pairs without losing information.



Example: A Supplier supplies a specific Part to a specific Project.

Relationship Sets

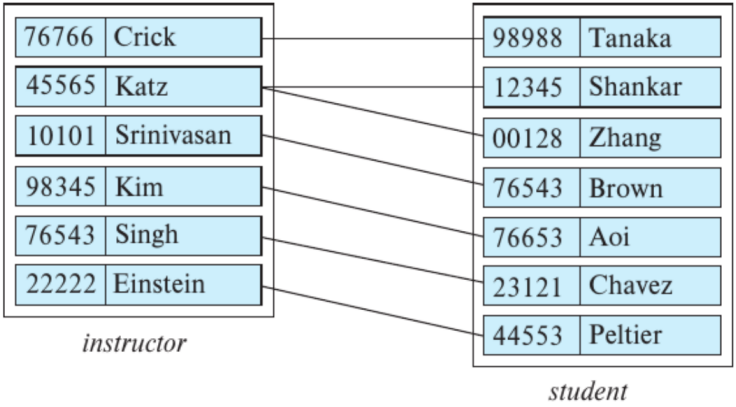


Figure: Relationship set: Example

Example: The ”Advisor” Relationship

We define the `Advisor` relationship set to denote the association between `Students` and the `Instructors` who act as their advisors.

- **Entity A**: Instructor (e.g., Einstein, Katz)
- **Entity B**: Student (e.g., Peltier, Shankar)
- **Relationship**: Advisor

Pictorially represented by a diamond connecting the two entity rectangles.

Represent Relationship Sets

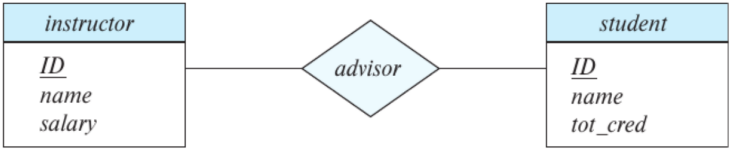


Figure: Diamond is the Relationship symbol

Relationship Sets: Additional Attribute

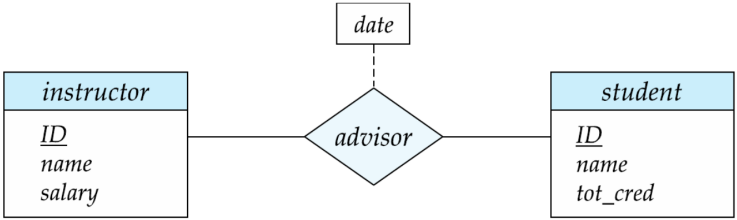


Figure: Additional Attribute in the Relationship

The attribute goes wherever the Foreign Key goes. In a database, the Foreign Key is the mechanism that creates that connection. Therefore, to correctly associate the attribute with the specific link it describes, you must place it in the same location as the Foreign Key.

Roles in Relationships

- Entity sets in a relationship do not need to be distinct.
- The same entity set may participate multiple times in different **roles**.
- This is also known as a **Recursive Relationship**.

Example: Course Prerequisite

- **Entity Set**: `Course`.
- **Relationship**: `Prereq`.
- **Role 1**: `course_id` (The main course).
- **Role 2**: `prereq_id` (The course required to take the main course).

Roles in Relationships

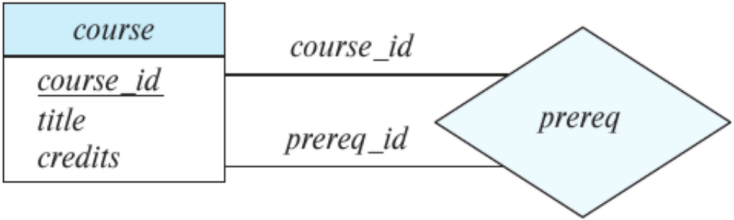


Figure: Roles in the Relationship

Degree of a Relationship Set

- The number of entity sets that participate in a relationship set is the degree of the relationship set.
- Most of the relationship sets in a database system are binary.
- Occasionally, however, relationship sets involve more than two entity sets.
- **Example:** Consider the entity projects that represents all the research projects of theuniversity. Each project can have multiple associated students and multiple associated instructors. It becomes a Ternary Relationship

Degree of a Relationship Set

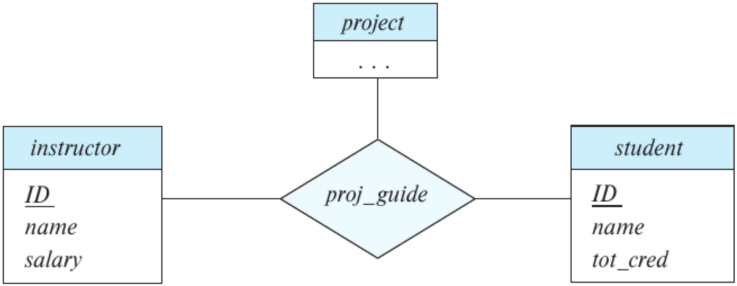


Figure: 3 Entities in Relationship

Attributes on Relationships

- Attributes can belong to a relationship set, not just entity sets.
- This is useful for describing the property of the association itself.

Example: Advisor Date

- **Attribute**: `Date`.
- **Context**: Tracks *when* the student started being associated with that specific advisor.
- This attribute cannot belong solely to the Student (who might change advisors) or the Instructor (who advises many students).

Mapping Cardinality

Mapping Cardinality Types

Cardinality expresses the number of entities to which another entity can be associated via a relationship set.

1. **One-to-One**
2. **One-to-Many**
3. **Many-to-One**
4. **Many-to-Many**

Example: One-to-One

An entity in A is associated with at most one in B, and vice versa
Real-life Example:

- **Entities**: `Person`, `Driving License`.
- **Logic**: A person holds one driving license; a driving license belongs to one person.

ER Notation: Arrows pointing to *both* entity sets.

Example: One-to-One

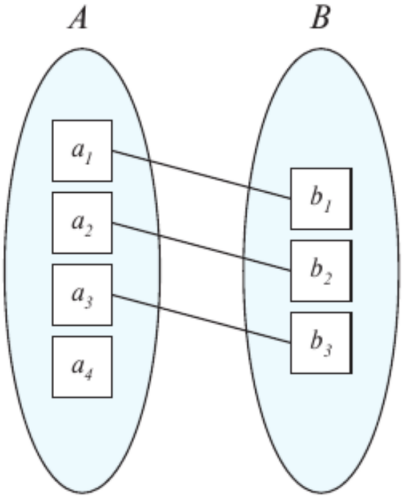


Figure: One to One Mapping

Example: One-to-Many

An entity in A is associated with any number of entities in B. An entity in B is associated with at most one entity in A.
Scenario: `Department` and `Student`.

- One Department has many Students.
- One Student belongs to only one Department.

ER Notation: Arrow pointing to `Department` (the "One" side). Line to `Student` (the "Many" side).

Example: One-to-Many

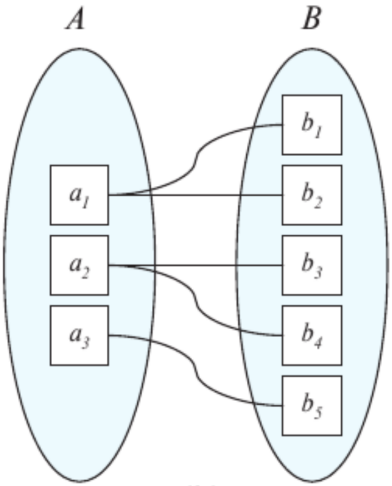


Figure: One to Many Mapping

Example: Many-to-Many

Entities in A and B can be associated with any number of entities in the other set.

Scenario: `Student` and `Course`.

- A Student takes many Courses.
- A Course is taken by many Students.

ER Notation: Simple lines (no arrows) connecting both entities.

Example: Many-to-Many

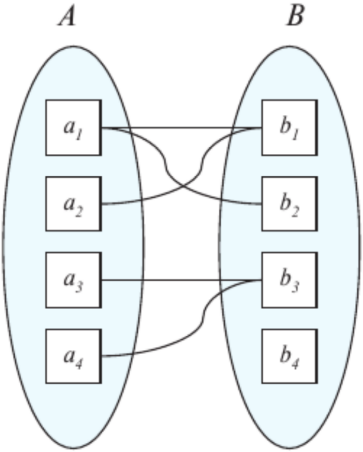


Figure: Many to Many Mapping

Recap: What is a Foreign Key?

- **Definition**: A Foreign Key (FK) is a column that creates a link between two tables.
- **Function**: It stores the **Primary Key** of another table to establish a relationship.
- **The Golden Rule**: A standard database cell (column) can hold only **one atomic value** (e.g., one number, one string).

Let's see how this works in a standard One-to-Many (1:N) scenario.

The "Normal" Case: One-to-Many (1:N)

Consider **Department (1)** → **Student (N)**.

- **Logic**: A Student belongs to *exactly one* Department.
- **Implementation**: We place the `Dept_ID` inside the `Student` table.

Department Table

D_ID	Name
10	CS
20	EEE

Student Table

ID	Name	Dept_ID (FK)
1	Alice	**10**
2	Bob	**10**
3	Charlie	**20**

Result: This works perfectly because every cell in the `Dept_ID` column contains just **one** number.

Step 1: The Independent Tables (Before Linking)

Let's look at our data *before* we try to create the relationship. We have two perfect, independent tables.

Table: Student

ID	Name
1	Alice
2	Bob

Table: Course

ID	Title
101	Math
102	Physics

The Goal: We need to record that **Alice (1)** takes **Math (101) AND Physics (102)** while bob takes Math(101).

Step 2: The Failed Implementation (1NF Violation)

If we try to use a simple Foreign Key (like we do in 1:N), we physically cannot store the data.

Option A: Add FK to Student?

ID	Name	Course_FK
1	Alice	**101, 102**
2	Bob	101

Fails: Cell contains multiple values.

Option B: Add FK to Course?

ID	Title	Student_FK
101	Math	**1, 2**
102	Phy	1

Fails: Cell contains multiple values.

*This violates **Atomicity** (First Normal Form). A column cannot hold a list.*

Step 3: The Solution (Junction Table)

Since neither table can hold the Foreign Key, we create a **third table** just to hold the connections.

Student

ID	Name
1	Alice
2	Bob

Takes (Junction)

S_ID	C_ID
1	101
1	102
2	101

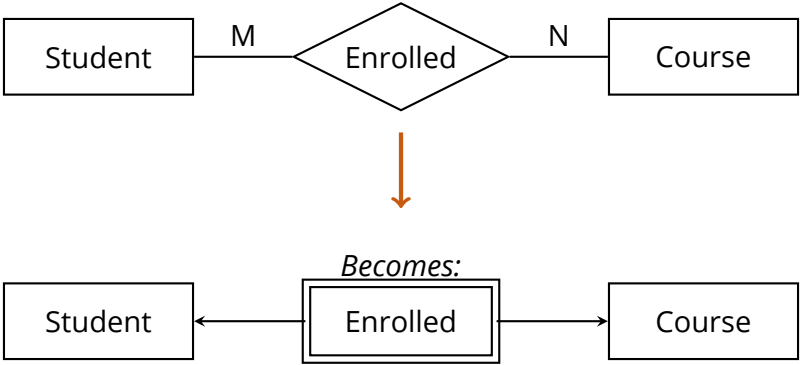
Course

ID	Title
101	Math
102	Phy

- Now, every cell has exactly **one** value.
- Alice (1) appears twice in the middle table, but that is allowed!

The Solution: The Junction Table

To resolve this, we break the **Many-to-Many** relationship into **two One-to-Many** relationships.
We create a new table (often called a *Junction*, *Bridge*, or *Association* table).



Junction Table

Structure of the Junction Table

The Junction Table contains the Primary Keys of *both* participating tables.

Student Table

S_ID	Name
1	Alice
2	Bob

Enrollment Table

S_ID	C_ID
1	**101**
1	**102**
2	**101**

Course Table

C_ID	Title
101	Math
102	Physics

Key Properties:

- `S_ID` is a Foreign Key referencing Student.
- `C_ID` is a Foreign Key referencing Course.
- The **Primary Key** of Enrollment is the combination `(S_ID, C_ID)`.

Handling Relationship Attributes

Sometimes, the relationship itself has data (e.g., `Grade`).

- If Alice gets an 'A' in Math, where do we store it?
- Not in `Student` (Alice has many grades).
- Not in `Course` (Math has many grades).

It must be stored in the Junction Table:

S_ID (FK)	C_ID (FK)	Grade
1	101	A
1	102	B+
2	101	A-

Participation Constraints

- **Total Participation**:
  - Every entity in the entity set *must* participate in the relationship.
  - **Example**: Every `Student` must have an `Advisor`.
  - **Notation**: Double Line.
- **Partial Participation**:
  - Some entities may not participate.
  - **Example**: Some `Instructors` might not advise any students.
  - **Notation**: Single Line.

Cardinality Constraints in ER Diagram

- **Visual Notation**:
  - A **directed line** ($\rightarrow$) is used to specify "one".
  - An **undirected line** (—) specifies "many" (zero or more).
- **Example: One-to-One**:
  - Relationship between an `instructor` and a `student`.
  - A student is associated with **at most one** instructor via the relationship `advisor`.

Cardinality Constraints in ER Diagram

Figure: One to One Mapping

ER Diagram for One-to-Many

- Consider the relationship between an `instructor` and a `student`.
- **Logic**:
  - An instructor is associated with **several** (including 0) students via `advisor`.
  - A student is associated with **at most one** instructor via `advisor`.

ER Diagram for One-to-Many

Figure: One to Many Mapping

ER Diagram for Many-to-One

- This is the reverse of the previous case.
- **Logic**:
  - An instructor is associated with **at most one** student via `advisor`.
  - A student is associated with **several** (including 0) instructors via `advisor`.

ER Diagram for Many-to-One

Figure: Many to One Mapping

ER Diagram for Many-to-Many

- Suppose we allow **joint supervision** where students work in a group.
- **Logic**:
  - An instructor is associated with **several** (possibly 0) students via `advisor`.
  - A student is associated with **several** (possibly 0) instructors via `advisor`.

ER Diagram for Many-to-Many

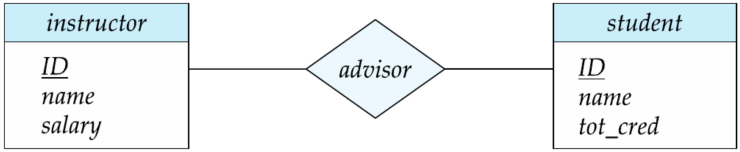


Figure: Many to Many Mapping

Total and Partial Participation

- **Total Participation** (Double Line):
  - Every entity in the entity set **must** participate in at least one relationship in the relationship set.
  - *Example*: Participation of `student` in `advisor` is total implies every student must have an instructor.
  - **Note**: Often implemented using `NOT NULL` Foreign Keys.
- **Partial Participation** (Single Line):
  - Some entities may not participate in any relationship.
  - *Example*: An instructor might not supervise any students.

Total and Partial Participation

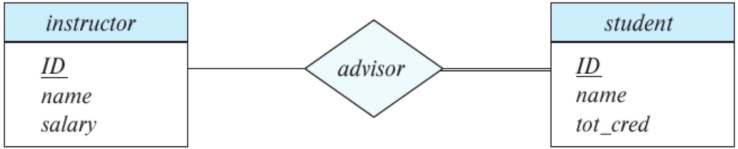


Figure: Total Participation

Max..Min Cardinality: Alternative Representation

- Used to indicate more complex constraints on participation frequency.
- **Notation**: Shown as $l..h$ on the line.
  - l is the **minimum** cardinality.
  - h is the **maximum** cardinality.
- **Key Rules**:
  - A minimum value of **1** indicates **total participation**.
  - $0..*$ means zero or more.

Max..Min Cardinality: Precautions

This format can be misleading if not read carefully. Consider the labels on the relationship line:

- Label **0..*** on the `Instructor` side:
  - Indicates an instructor can have zero or more students.
- Label **1..1** on the `Student` side:
  - Indicates a student must have **exactly one** instructor.
- **Result**: The relationship is One-to-Many from Instructor to Student, with total participation from Student.

Max..Min Cardinality: Precautions

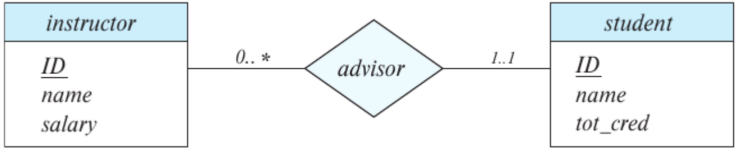


Figure: Min..Max format in cardinality

Weak and Strong Entity Sets

Weak and Strong Entity Sets

- Strong Entity is independent of any other entity in the schema, in other words, it has sufficient attributes to form the primary key.
- The strong entity is represented by a single rectangle.
- The relationship between two strong entities is represented by a single diamond.
- A weak entity is an entity set that does not have sufficient attributes for Unique Identification of its records. (no primary key)

Weak and Strong Entity Sets

- It has no primary key. The existence of a weak entity set depends on the existence of a strong entity set
- It has partial key, which is combined with the primary key of the its strong entity set on the other-side of the relation to distinguish each record.
- Note: In most cases the design is flawed, a better design is possible where both will be strong entity sets.

Weak and Strong Entity Sets

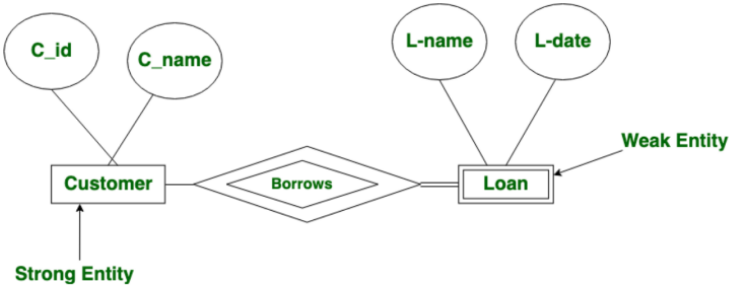


Figure: Weak Entity Set[1]

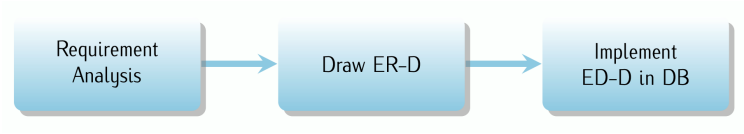
ER Diagram Implementation

The Conversion Process

The design process flows through specific stages to move from requirements to a working database:

1. **Requirement Analysis**: Understanding what data is needed.
2. **Draw ER-D**: Modeling the conceptual design.
3. **Implement ER-D in DB**: Converting the diagram into tables (DDL).

Note on Mappings: In theory, there are 4 types of mapping cardinalities. However, we can eliminate one in practice because **One-to-Many** and **Many-to-One** are identical if we simply change the direction of the relationship.


Requirement Analysis
Draw ER-D
Implement ED-D in DB

Real-life Mapping Examples

Before implementing, we categorize relationships found in the real world:

- **One to One**:
  - *Example*: Passport and Driving License (assuming a strict 1:1 link for verification).
- **One to Many**:
  - *Example*: Department and Students (One dept has many students).
- **Many to Many**:
  - *Example*: Students and Courses (Students take many courses; courses have many students).

Handling Redundant Attributes

Sometimes an ER diagram includes attributes that replicate information already defined by a relationship.

Example Scenario:

- Entity `student` has attributes: `ID`, `name`, `tot_cred`, and **dept_name**.
- Entity `department` has attributes: `dept_name`, `building`, `budget`.
- Relationship `stud_dept` connects them.

The Issue: The attribute `dept_name` inside `student` is redundant because the relationship `stud_dept` already establishes the link. It should be removed from the ER model.

Note: When converting back to tables, this attribute often gets reintroduced as a Foreign Key.

Handling Redundant Attributes

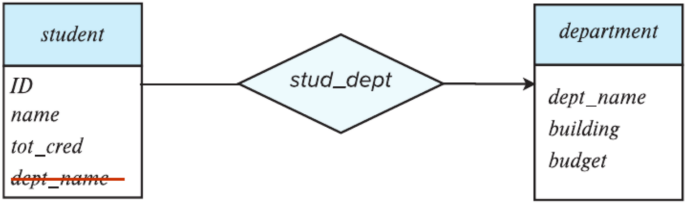


Figure: Redundant Attribute in ER-D

Implementation Rules: 1:N and 1:1

- **One to Many (1:N)**:
  - In the table representing the ”Many” side, use a **Foreign Key** referencing the first entity (”One” side).
- **One to One (1:1)**:
  - Can be implemented in either table (depending on context).
  - Use a **Foreign Key** pointing to the other entity.
  - **Constraint**: You must apply a `UNIQUE` constraint to this Foreign Key to enforce 1:1 cardinality.

Implementation Rules: M:N and Participation

- **Many to Many (M:N)**:
  - Requires a third entity (table) termed as a **Junction Table**.
  - This table is formed by the Foreign Keys of both participating entities.
  - Additional relationship attributes (e.g., *grade*) appear here.
- **Total Participation**:
  - For total participation in the second entity (the side holding the FK), use a `NOT NULL` constraint along with the Foreign Key.
  - **Warning**: Total participation implies strict business logic which may be impractical in some real-world cases.

Implementation Practice: IUT Scenario

Scenario:

- IUT has a number of **departments**.
- In each department, **students** are admitted.
- Each student must have an ID, Name, and Address.
- In each semester, a number of **courses** are offered.
- A student can take a number of courses.

Task: Draw the ER-D for this scenario and implement it using SQL DDLs.

Constraints on Generalization/Specialization

Condition-defined vs. User-defined

Note: In ER-D, specialization is depicted by a hollow arrow-head pointing to the generalization entity.

- **Condition-defined**:
  - Membership is evaluated based on the value of an attribute in the higher-level entity.
  - *Example*: All `customers` over 65 years are members of the `senior-citizen` entity set.
  - Since membership is based on the `age` attribute, it is attribute-defined.
- **User-defined**:
  - Not constrained by a membership condition.
  - The database user explicitly assigns entities to a given entity set.
  - *Example*: After 3 months, university employees are manually assigned to one of four specific work teams.

Generalization and Specialization

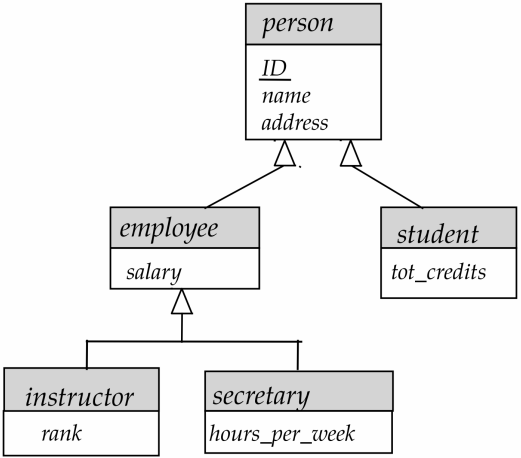
Generalization Concepts

- **Specialization**: Top-down. Divides an entity into subgroups.
- **Generalization**: Bottom-up (IS-A relationship). Combines entities with shared features.
- Represented by a hollow arrow-head pointing to the higher-level entity (ISA relationship).

Generalization in ER Models

- **Definition**: A design process that proceeds in a *bottom-up* manner.
- It combines a number of entity sets that share the same features into a higher-level entity set.
- **Key Concept**: The ”IS-A” Relationship.
- **Representation**:
  - Denoted by a triangle component labeled **IS-A** (or simply a triangle).
  - Represents the relationship between a *superclass* (higher level) and *subclasses* (lower level).

Example: Person Hierarchy



Analyzing the Hierarchy

Referencing the provided diagram structure:

- **Level 1 (Root):** `Person`
  - Represents the most general entity.
  - Contains attributes common to all humans in the system (`ID`, `name`, `address`).
- **Level 2 (Branching):** `Employee` and `Student`
  - Both represent people, but they differ in role.
  - `Employee` adds `salary`.
  - `Student` adds `tot_credits`.
- **Level 3 (Specialization):** `Instructor` and `Secretary`
  - Both are employees, but have specific job details.

Attribute Inheritance

A crucial property of generalization is **Inheritance**.

The Rule:

- Attributes of higher-level entity sets are inherited by lower-level entity sets.
- A subclass has its *own* attributes + *all* attributes of its superclass.

Example: The Instructor Entity

- **Explicit Attribute**: `rank` (defined in `instructor` box).
- **Inherited Level 1**: `salary` (from `employee`).
- **Inherited Level 2**: `ID, name, address` (from `person`).
- **Total Attributes**: An instructor has **5** attributes in total.

Why use Generalization?

1. Avoids Redundancy

Without generalization, we would have to repeat common attributes like `name` and `address` in the `Student`, `Instructor`, and `Secretary` tables separately.

2. Modularity

If we need to change the format of an address, we only modify the `Person` entity. The change automatically reflects for all subclasses conceptually.

Implementation Note

When converting to SQL tables, the `ID` (Primary Key) typically travels down to the subclasses to link the rows together.

Implementing Generalization in SQL

The Challenge

Standard SQL does not have a `GENERALIZATION` keyword. We must simulate the "IS-A" hierarchy manually.

The Strategy: Joined Tables

We create a separate table for each entity in the hierarchy.

1. **Superclass Table**: Contains common attributes.
2. **Subclass Table**: Contains specific attributes + Primary Key.
3. **The Link**: The Subclass Primary Key also acts as a **Foreign Key** pointing to the Superclass.

Step 1 & 2: The Top Levels

First, we create the root (`Person`) and the first child (`Employee`).

```sql
-- 1. The Root (Superclass)
CREATE TABLE Person (
person_id INT PRIMARY KEY,
name VARCHAR(100),
address VARCHAR(255)
);

-- 2. The Subclass
CREATE TABLE Employee (
person_id INT PRIMARY KEY, -- PK for this table
salary DECIMAL(10, 2),

-- Links back to Person
FOREIGN KEY (person_id) REFERENCES Person(person_id)
);
```

Step 3: The Deepest Subclass

The `Instructor` table links to its *immediate* parent, `Employee`, not the root.

```sql
-- 3. The Leaf Node
CREATE TABLE Instructor (
person_id INT PRIMARY KEY,
rank VARCHAR(50),

-- Links to Employee (Immediate Parent)
FOREIGN KEY (person_id) REFERENCES Employee(person_id)
);
```

Key Observation

The `person_id` travels down the entire chain. To identify an instructor, the same ID exists in all three tables.

Handling Data: Insertion & Retrieval

Insertion: Data must be inserted layer-by-layer.

```sql
-- Insert into Person first, then Employee, then Instructor
INSERT INTO Person VALUES (101, 'Akib, 'Dhaka');
INSERT INTO Employee VALUES (101, 50000);
INSERT INTO Instructor VALUES (101, 'Junior Lecturer');
```

Retrieval: We must `JOIN` to reconstruct the object.

```sql
SELECT p.name, e.salary, i.rank
FROM Person p
JOIN Employee e ON p.person_id = e.person_id
JOIN Instructor i ON e.person_id = i.person_id
WHERE p.person_id = 101;
```

Method 2: The "No Superclass" Strategy

In Method 1, we kept a separate table for the parent entity (`Person`). In Method 2, we **remove** the higher-level entity set table entirely.

How it works

We create tables *only* for the lower-level entity sets (subclasses).

- The attributes of the higher-level entity (e.g., `name, street, city`) are copied down into each subclass table.
- There is NO link or foreign key pointing "upwards" because the upper table does not exist.

Schema Result

Instead of 3 tables, we get 2:

- **employee** (<u>ID</u>, name, street, city, salary)
- **student** (<u>ID</u>, name, street, city, totalcredit)

When can we use Method 2?

This method is NOT always safe. It can only be applied if two specific constraints are met:

1. **Disjoint Constraint**:
   - An entity cannot belong to two lower-level sets at once.
   - *Why?*: If a person was both Employee and Student, we would store their `name` and `address` twice (redundancy).
2. **Total (Complete) Participation**:
   - Every entity in the higher-level set MUST belong to a lower-level set.
   - *Why?*: Since there is no `Person` table, where would we store a "just a Person" who is neither student nor employee? We would lose them.

Strengths and Weaknesses

Strength: Speed

- **No Joins Required**: All data for an employee is in one single row.
- Access is faster than Method 1 because we don't need to combine tables to find a name or address.

Weaknesses

- **Data Integrity Issues**: Since there is no central `Person` table, we cannot easily enforce global uniqueness for IDs across students and employees.
- **Redundancy Risk**: If the generalization is overlapping (e.g., a TA is both Student and Employee), attributes like `street` and `city` are stored twice.

Constraints on Membership

- **Condition-defined**: Membership based on attribute value.
  - *Example*: All customers with age > 65 are "Senior Citizens".
- **User-defined**: Manual assignment.
  - *Example*: Assigning employees to specific work teams.
- **Disjoint**: Entity belongs to max one subgroup.
  - *Example*: A Person is either Student OR Employee.
- **Overlapping**: Entity belongs to multiple subgroups.
  - *Example*: A Person is both Student AND Employee (e.g., a Teaching Assistant).

Implementing ER Diagrams in Tables

Converting Strong and Weak Entities

Strong Entity:

- Create a table with all simple attributes.
- Entity Primary Key (PK) becomes Table PK.

Weak Entity:

- Create a table with all attributes.
- Add the Owner's PK as a Foreign Key (FK).
- **Table PK** = Owner's PK + Partial Key.

Converting Relationships (1:N and 1:1)

One-to-Many:

- Identify the entity on the ”Many” side.
- Add the PK of the ”One” side entity to the ”Many” side table as a Foreign Key.

One-to-One:

- Choose one table (often the one with total participation).
- Add the PK of the other entity as a Foreign Key.
- Apply a `UNIQUE` constraint to the FK to ensure 1:1 mapping.

Converting Relationships (Many-to-Many)

For M:N relationships, we cannot simply add columns to existing tables.
Junction Table:

- Create a new table (e.g., `Student_Courses`).
- Include the Primary Keys of both participating entities as Foreign Keys.
- Include any descriptive attributes of the relationship (e.g., `Grade`).
- **Primary Key**: The combination of the two Foreign Keys.

Questions?